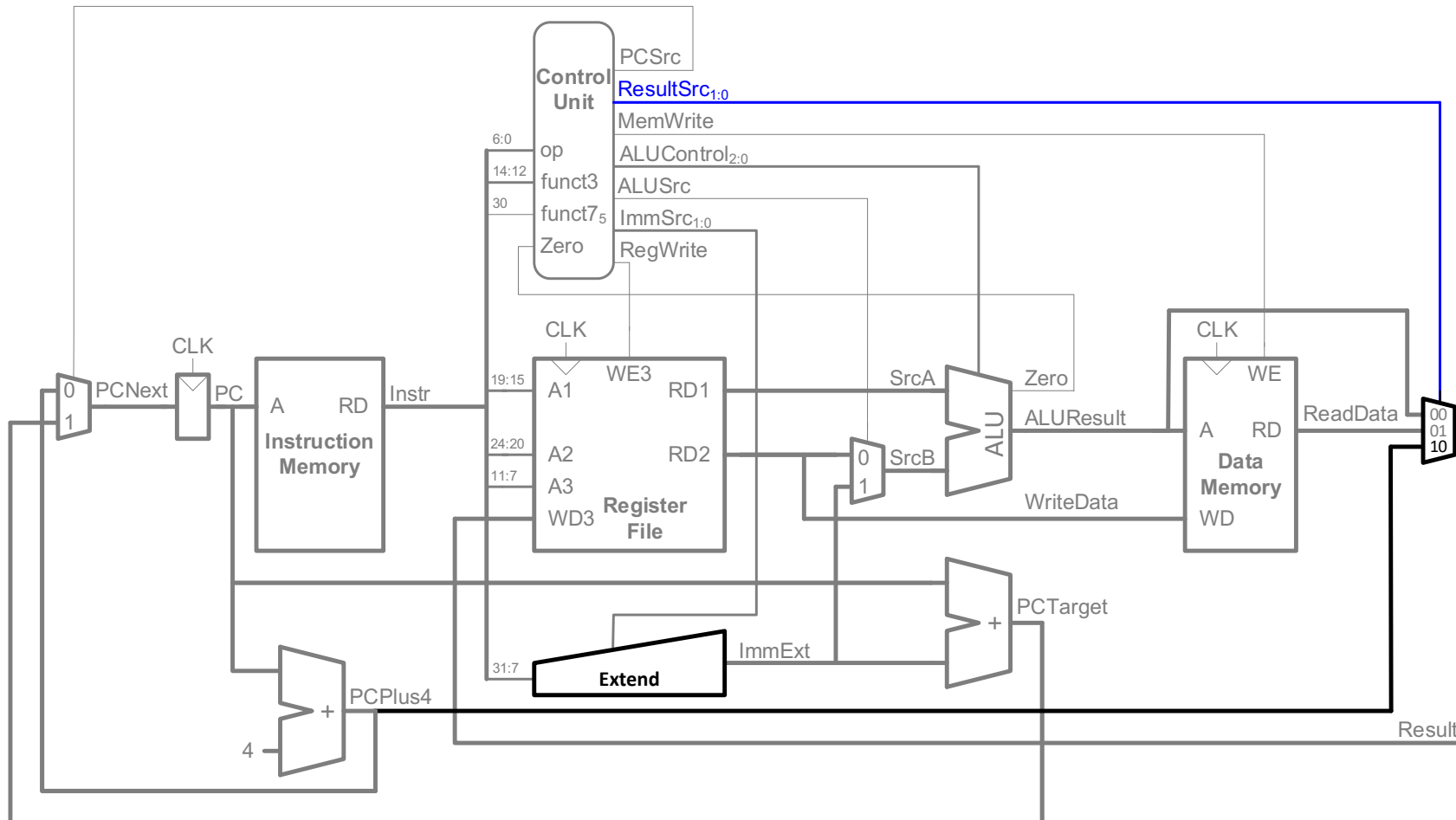


Verilog Implementation

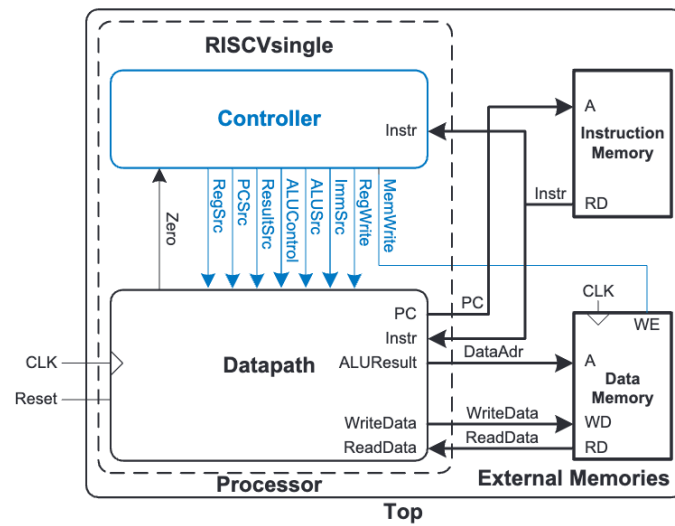
Single Cycle DataPath



Control Unit Table

op	Instruct	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
3	lw	1	00	1	0	01	0	00	0
35	sw	0	01	1	1	XX	0	00	0
51	R-type	1	XX	0	0	00	0	10	0
99	beq	0	10	0	0	XX	1	01	0
19	I-type	1	00	1	0	00	0	10	0
111	jal	1	11	X	0	10	0	XX	1

Figure 7.63 Single-cycle processor interfaced to external memories



Single Cycle

Instruction Memory and Data Memory

HDL Example 7.15 DATA MEMORY

SystemVerilog

```
module dmem(input  logic      clk, we,  
            input  logic [31:0] a, wd,  
            output logic [31:0] rd);  
  
    logic [31:0] RAM[63:0];  
  
    assign rd = RAM[a[31:2]]; // word aligned  
  
    always_ff @(posedge clk)  
        if (we) RAM[a[31:2]] <= wd;  
endmodule
```

HDL Example 7.14 INSTRUCTION MEMORY

SystemVerilog

```
module imem(input  logic [31:0] a,  
            output logic [31:0] rd);  
  
    logic [31:0] RAM[63:0];  
  
    initial  
        $readmemh("riscvtest.txt",RAM);  
  
    assign rd = RAM[a[31:2]]; // word aligned  
endmodule
```

Example of Machine Code Loaded on the IM

# riscvtest.s					
# Sarah.Harris@unlv.edu					
# David_Harris@hmc.edu					
# 27 Oct 2020					00500113
#					00C00193
# Test the RISC-V processor:					FF718393
# add, sub, and, or, slt, addi, lw, sw, beq, jal					0023E233
# If successful, it should write the value 25 to address 100					0041F2B3
#	RISC-V Assembly	Description	Address	Machine Code	
main:	addi x2, x0, 5	# x2 = 5	0	00500113	004282B3
	addi x3, x0, 12	# x3 = 12	4	00C00193	02728863
	addi x7, x3, -9	# x7 = (12 - 9) = 3	8	FF718393	0041A233
	or x4, x7, x2	# x4 = (3 OR 5) = 7	C	0023E233	00020463
	and x5, x3, x4	# x5 = (12 AND 7) = 4	10	0041F2B3	004282B3
	add x5, x5, x4	# x5 = 4 + 7 = 11	14	004282B3	00000293
	beq x5, x7, end	# shouldn't be taken	18	02728863	0023A233
	slt x4, x3, x4	# x4 = (12 < 7) = 0	1C	0041A233	005203B3
	beq x4, x0, around	# should be taken	20	00020463	00000293
	addi x5, x0, 0	# shouldn't execute	24	00000293	402383B3
around:	slt x4, x7, x2	# x4 = (3 < 5) = 1	28	0023A233	0471AA23
	add x7, x4, x5	# x7 = (1 + 11) = 12	2C	005203B3	06002103
	sub x7, x7, x2	# x7 = (12 - 5) = 7	30	402383B3	005104B3
	sw x7, 84(x3)	# [96] = 7	34	0471AA23	008001EF
	lw x2, 96(x0)	# x2 = [96] = 7	38	06002103	00100113
	add x9, x2, x5	# x9 = (7 + 11) = 18	3C	005104B3	00910133
	jal x3, end	# jump to end, x3 = 0x44	40	008001EF	00100113
	addi x2, x0, 1	# shouldn't execute	44	00100113	00910133
end:	add x2, x2, x9	# x2 = (7 + 18) = 25	48	00910133	0221A023
	sw x2, 0x20(x3)	# [100] = 25	4C	0221A023	00210063
done:	beq x2, x2, done	# infinite loop	50	00210063	

Figure 7.64 riscvtest.s

Figure 7.65 riscvtest.txt

Controller

HDL Example 7.2 CONTROLLER

SystemVerilog

```
module controller(input logic [6:0] op,
                  input logic [2:0] funct3,
                  input logic      funct7b5,
                  input logic      Zero,
                  output logic [1:0] ResultSrc,
                  output logic      MemWrite,
                  output logic      PCSrc, ALUSrc,
                  output logic      RegWrite, Jump,
                  output logic [1:0] ImmSrc,
                  output logic [2:0] ALUControl);

  logic [1:0] ALUOp;
  logic      Branch;

  maindec md(op, ResultSrc, MemWrite, Branch,
             ALUSrc, RegWrite, Jump, ImmSrc, ALUOp);
  aludec ad(op[5], funct3, funct7b5, ALUOp, ALUControl);

  assign PCSrc = Branch & Zero | Jump;
endmodule
```

HDL Example 7.3 MAIN DECODER

SystemVerilog

```
module maindec(input logic [6:0] op,
               output logic [1:0] ResultSrc,
               output logic      MemWrite,
               output logic      Branch, ALUSrc,
               output logic      RegWrite, Jump,
               output logic [1:0] ImmSrc,
               output logic [1:0] ALUOp);

  logic [10:0] controls;

  assign {RegWrite, ImmSrc, ALUSrc, MemWrite,
         ResultSrc, Branch, ALUOp, Jump} = controls;

  always_comb
  case(op)
    // RegWrite_ImmSrc_ALUSrc_MemWrite_ResultSrc_Branch_ALUOp_Jump
    7'b0000011: controls = 11'b1_00_1_0_01_0_00_0; // lw
    7'b0100011: controls = 11'b0_01_1_1_00_0_00_0; // sw
    7'b0110011: controls = 11'b1_xx_0_0_00_0_10_0; // R-type
    7'b1100011: controls = 11'b0_10_0_0_00_1_01_0; // beq
    7'b0010011: controls = 11'b1_00_1_0_00_0_10_0; // I-type ALU
    7'b1101111: controls = 11'b1_11_0_0_10_0_00_1; // jal
    default:    controls = 11'bx_xx_x_x_xx_x_xx_x; // ???
  endcase
endmodule
```

HDL Example 7.4 ALU DECODER

SystemVerilog

```
module aludec(input logic      opb5,
              input logic [2:0] funct3,
              input logic      funct7b5,
              input logic [1:0] ALUOp,
              output logic [2:0] ALUControl);

  logic RtypeSub;
  assign RtypeSub = funct7b5 & opb5; // TRUE for R-type subtract

  always_comb
  case(ALUOp)
    2'b00:      ALUControl = 3'b000; // addition
    2'b01:      ALUControl = 3'b001; // subtraction
    default: case(funct3) // R-type or I-type ALU
      3'b000: if (RtypeSub)
                ALUControl = 3'b001; // sub
              else
                ALUControl = 3'b000; // add, addi
      3'b010: ALUControl = 3'b101; // slt, slti
      3'b110: ALUControl = 3'b011; // or, ori
      3'b111: ALUControl = 3'b010; // and, andi
      default: ALUControl = 3'bxxx; // ???
    endcase
  endcase
endmodule
```

ALU

```
module alu(input  [31:0] a, b, input  [2:0] alucont, output reg [31:0] result, output zero);
  wire [31:0] b2, sum, slt;
  assign b2 = alucont[2] ? ~b:b;
  assign sum = a + b2 + alucont[2];
  assign slt = sum[31];

  always@(*)
    case(alucont[1:0])
      2'b00: result <= a & b;
      2'b01: result <= a | b;
      2'b10: result <= sum;
      2'b11: result <= slt;
    endcase
  assign zero = (result == 32'b0);
endmodule
```


REGISTER FILE

```
module regfile(input      clk, input      we3, input [4:0] ra1, ra2, wa3, input [31:0] wd3, output [31:0]
rd1, rd2);
    reg [31:0] rf[31:0];
    // three ported register file, read two ports combinational
    // write third port on rising edge of clock, register 0 hardwired to 0
    always @(posedge clk)
        if (we3) rf[wa3] <= wd3;
    assign rd1 = (ra1 != 0) ? rf[ra1] : 0;
    assign rd2 = (ra2 != 0) ? rf[ra2] : 0;
endmodule
```

Single Cycle Data Path

HDL Example 7.5 DATAPATH

SystemVerilog

```
module datapath(input  logic      clk, reset,
               input  logic [1:0] ResultSrc,
               input  logic      PCSrc, ALUSrc,
               input  logic      RegWrite,
               input  logic [1:0] ImmSrc,
               input  logic [2:0] ALUControl,
               output logic      Zero,
               output logic [31:0] PC,
               input  logic [31:0] Instr,
               output logic [31:0] ALUResult, WriteData,
               input  logic [31:0] ReadData);

  logic [31:0] PCNext, PCPlus4, PCTarget;
  logic [31:0] ImmExt;
  logic [31:0] SrcA, SrcB;
  logic [31:0] Result;

  // next PC logic
  flopr #(32) pcreg(clk, reset, PCNext, PC);
  adder      pcadd4(PC, 32'd4, PCPlus4);
  adder      pcaddbranch(PC, ImmExt, PCTarget);
  mux2 #(32) pcmux(PCPlus4, PCTarget, PCSrc, PCNext);

  // register file logic
  regfile    rf(clk, RegWrite, Instr[19:15], Instr[24:20],
               Instr[11:7], Result, SrcA, WriteData);
  extend     ext(Instr[31:7], ImmSrc, ImmExt);

  // ALU logic
  mux2 #(32) srcbmux(WriteData, ImmExt, ALUSrc, SrcB);
  alu        alu(SrcA, SrcB, ALUControl, ALUResult, Zero);
  mux3 #(32) resultmux(ALUResult, ReadData, PCPlus4,
                      ResultSrc, Result);
endmodule
```

SystemVerilog

```
module top(input  logic      clk, reset,
           output logic [31:0] WriteData, DataAdr,
           output logic      MemWrite);

  logic [31:0] PC, Instr, ReadData;

  // instantiate processor and memories
  riscvsingle rvsingle(clk, reset, PC, Instr, MemWrite,
                      DataAdr, WriteData, ReadData);
  imem imem(PC, Instr);
  dmem dmem(clk, MemWrite, DataAdr, WriteData, ReadData);
endmodule
```



RISC-V Single Cycle

HDL Example 7.1 SINGLE-CYCLE PROCESSOR

SystemVerilog

```
module riscvsingle(input  logic      clk, reset,
                  output logic [31:0] PC,
                  input  logic [31:0] Instr,
                  output logic      MemWrite,
                  output logic [31:0] ALUResult, WriteData,
                  input  logic [31:0] ReadData);

    logic      ALUSrc, RegWrite, Jump, Zero;
    logic [1:0] ResultSrc, ImmSrc;
    logic [2:0] ALUControl;

    controller c(Instr[6:0], Instr[14:12], Instr[30], Zero,
                ResultSrc, MemWrite, PCSrc,
                ALUSrc, RegWrite, Jump,
                ImmSrc, ALUControl);
    datapath dp(clk, reset, ResultSrc, PCSrc,
                ALUSrc, RegWrite,
                ImmSrc, ALUControl,
                Zero, PC, Instr,
                ALUResult, WriteData, ReadData);

endmodule
```

Testing Single Cycle

SystemVerilog

```
module testbench();

    logic        clk;
    logic        reset;
    logic [31:0] WriteData, DataAdr;
    logic        MemWrite;

    // instantiate device to be tested
    top dut(clk, reset, WriteData, DataAdr, MemWrite);

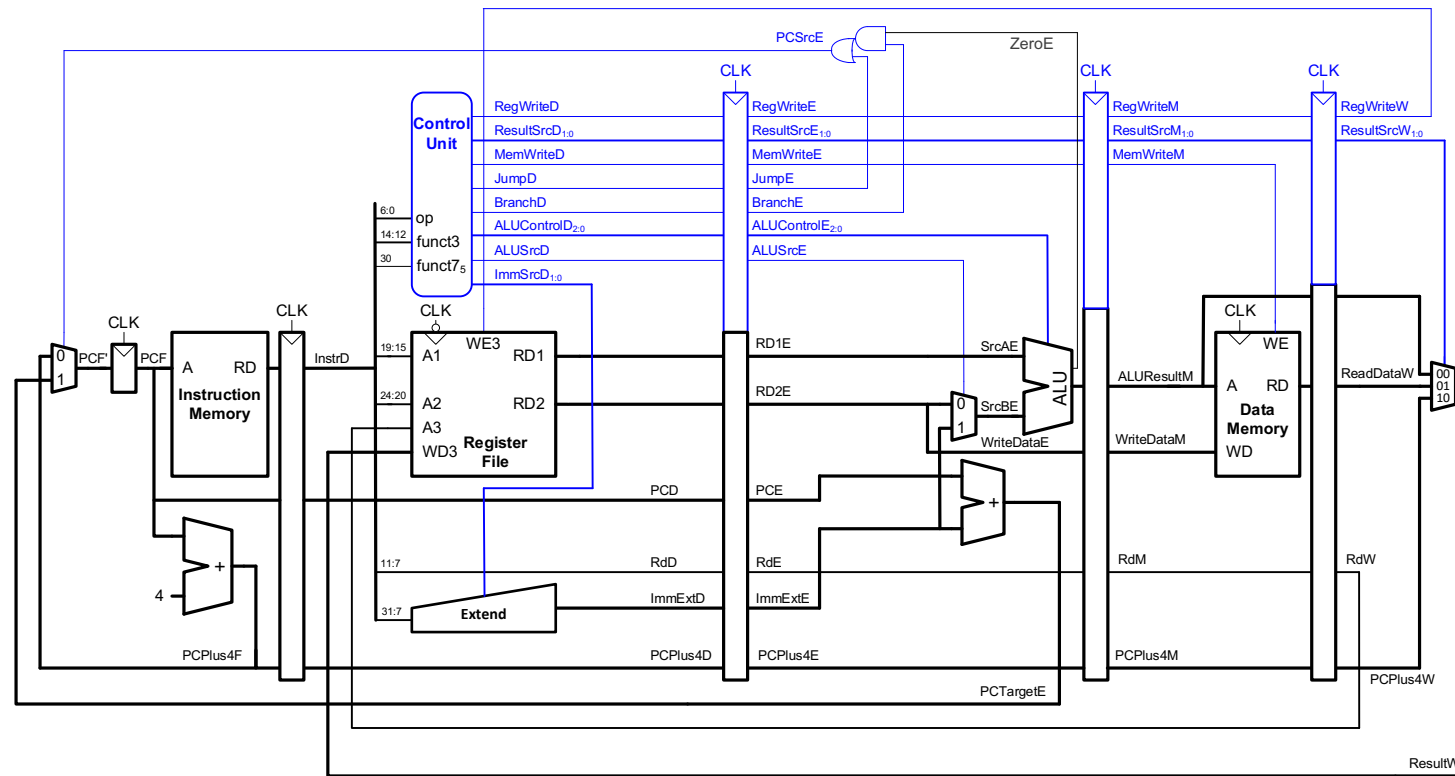
    // initialize test
    initial
    begin
        reset <= 1; # 22; reset <= 0;
    end

    // generate clock to sequence tests
    always
    begin
        clk <= 1; # 5; clk <= 0; # 5;
    end

    // check results
    always @(negedge clk)
    begin
        if(MemWrite) begin
            if(DataAdr === 100 & WriteData === 25) begin
                $display("Simulation succeeded");
                $stop;
            end else if (DataAdr !== 96) begin
                $display("Simulation failed");
                $stop;
            end
        end
    end
endmodule
```

Pipeline

Pipeline without Hazard Detection



PC Register and Pipeline Registers

```
module floprc #(parameter WIDTH = 8)
```

```
    (input          clk, reset, clear,
```

```
     input  [WIDTH-1:0] d,
```

```
     output reg [WIDTH-1:0] q);
```

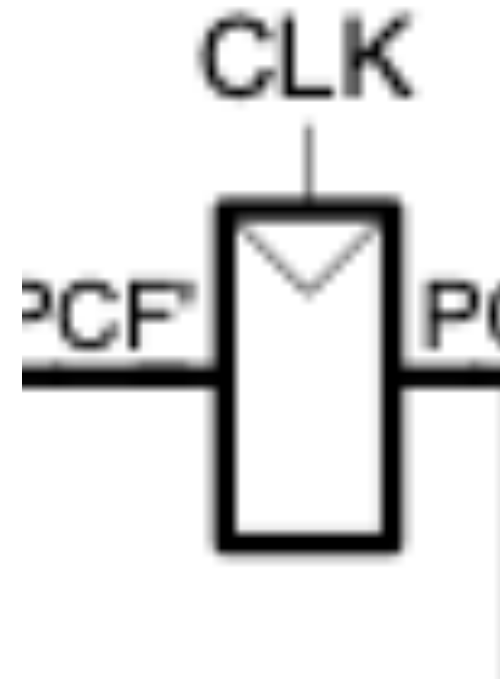
```
always @(posedge clk, posedge reset)
```

```
    if (reset)    q <= #1 0;
```

```
    else if (clear) q <= #1 0;
```

```
    else          q <= #1 d;
```

```
endmodule
```



Pipelined DataPath

```
module datapath(input      clk, reset,  
                ...);  
  
    // register file (operates in decode and writeback)  
    regfile    rf(clk, regwriteW, rsD, rtD, writeregW,  
                  resultW, srcaD, srcbD);  
  
    // Fetch stage logic  
    flopenr #(32) pcreg(clk, reset, ~stallF, pcnextFD, pcF);  
    adder    pcadd1(pcF, 32'b100, pcplus4F);  
    ...
```


Main Control Pipelined

```
module controller(input    clk, reset,..., output [2:0] alucontrolE);
    maindec md(opD, memtoregD, memwriteD, branchD,alusrcD, regdstD, regwriteD, jumpD,aluopD);

    aludec ad(functD, aluopD, alucontrolD);

    assign pcsrcD = branchD & equalD;
    // pipeline registers

    floprc #(8) regE(clk, reset, flushE,
        {memtoregD, memwriteD, alusrcD, regdstD, regwriteD, alucontrolD},
        {memtoregE, memwriteE, alusrcE, regdstE, regwriteE, alucontrolE});
    flopr #(3) regM(clk, reset,
        {memtoregE, memwriteE, regwriteE},
        {memtoregM, memwriteM, regwriteM});
    flopr #(2) regW(clk, reset,
        {memtoregM, regwriteM},
        {memtoregW, regwriteW});
endmodule
```